

PROJECT 2: MODEL ANALYSIS PROJECT

- **Vision:** Programming is more than writing code. The ultimate goal of the projects in this course is that you learn to structure a programming project from start to finish. Your answers should of course be correct, but your code should also be well-structured and documented.
- **Objectives:** In your model analysis project, you should show that you can
 1. Apply numerical model analysis methods (optimization, root-finding, etc.)
 2. Present results in text form and in easy-to-understand figures
 3. Teach yourself new functionality in a package you already know
 4. Structure a code project
 5. Document code
- **Structure:** Your model analysis project should consist of
 1. A single self-contained notebook (.ipynb) presenting the analysis
 2. Documented Python module files (.py)

Starting point: Use the files `Consumer.py` and `Government.py`. Everything except the methods marked with `pass` is written for you.

Example of structure: See this [repository](#) and `ProgEcon-lectures/Exams/Structure/`
- **Hand-in:** Upload in Absalon
- **Deadline:** See [Calendar](#).
- **Feedback:** Your TA will provide feedback on your project.
- **Exam:** Your model analysis project will be a part of your exam portfolio. You can incorporate feedback before handing in the final version.
This first version does not need to be perfect!

1 A consumer with two nests

A consumer buys three goods in quantities $x_1, x_2, x_3 \geq 0$ at prices $p_1, p_2, p_3 > 0$, and has income $I > 0$.

Goods 2 and 3 belong together in a **nest**, while good 1 is on its own. Say good 1 is **food**, and goods 2 and 3 are **bus trips** and **train trips**. Bus and train are both ways of travelling, so they are much more like each other than either of them is like food. That is all the nest says.

First the two travel goods are combined into a **travel composite**

$$x_B = [\beta x_2^{\rho_B} + (1 - \beta)x_3^{\rho_B}]^{\frac{1}{\rho_B}}, \quad (1)$$

and then food and travel are combined into utility,

$$u(x_1, x_2, x_3) = [\alpha x_1^{\rho_A} + (1 - \alpha)x_B^{\rho_A}]^{\frac{1}{\rho_A}}, \quad (2)$$

where $\alpha \in (0, 1)$ is the weight on food and $\beta \in (0, 1)$ is the weight on the bus.

The two exponents are $\rho_A = 1 - \frac{1}{\sigma_A}$ and $\rho_B = 1 - \frac{1}{\sigma_B}$, where $\sigma_A > 0$ and $\sigma_B > 0$ say how easily the consumer can **substitute** one good for the other:

- $\sigma_B < 1$: bus and train are **complements**. The consumer wants both, and cannot get far by replacing one with the other.
- $\sigma_B > 1$: bus and train are **substitutes**. Either one will do, so the consumer mostly buys whichever is cheaper.

and the same for σ_A , between food and travel. We always use $\sigma \neq 1$, so that $\rho \neq 0$ and you never divide by zero in the exponent.

1.1 The problem in nested budget shares

Rather than in quantities, we write the problem in **budget shares**, $s_j = \frac{p_j x_j}{I}$, so that $x_j = \frac{s_j I}{p_j}$. Utility is increasing in all three goods, so the consumer never throws money away and spends everything, $s_1 + s_2 + s_3 = 1$.

We now describe the choice in the **same two steps as the nests**:

1. $s_1 \in [0, 1]$: the share of income spent on **food**. That leaves the share $1 - s_1$ for travel.

2. $w \in [0, 1]$: the share of the **travel budget** spent on the **bus**.

The three budget shares then follow as

$$s_1, \quad s_2 = (1 - s_1)w, \quad s_3 = (1 - s_1)(1 - w), \quad (3)$$

and the quantities as $x_j = \frac{s_j I}{p_j}$. The problem is

$$V = \max_{s_1 \in [0,1], w \in [0,1]} u \left(\frac{s_1 I}{p_1}, \frac{(1 - s_1)wI}{p_2}, \frac{(1 - s_1)(1 - w)I}{p_3} \right). \quad (4)$$

We call (s_1, w) the **nested budget shares**. They mirror the nests: the consumer first decides how much of the budget to spend on getting around at all, and then how to split that between the bus and the train.

Why this is the right way to write it. Three reasons, and the third is the important one:

1. The budget constraint holds exactly, by construction. Whatever (s_1, w) you pick, the three shares in eq. (3) sum to one, so no money is ever left on the table and none is overspent.
2. There are only **two** variables to choose instead of three.
3. **Every** point of the unit square is a possible choice, and every possible choice is a point of the unit square. The constraint set is therefore exactly the box $[0, 1] \times [0, 1]$ – and a box is precisely what L-BFGS-B takes as its bounds. So no penalties, no projections and no constraint handling of any kind are needed anywhere in this project.

Point 3 is worth dwelling on, because it is easy to get wrong. The obvious thing to do is to choose the shares s_1 and s_2 directly and let $s_3 = 1 - s_1 - s_2$ be the residual. But then $s_1 \geq 0$ and $s_2 \geq 0$ are bounds, while the third restriction $s_1 + s_2 \leq 1$ ties the two variables together and is not a bound at all: the possible choices are a **triangle**, and a solver that only understands boxes does not know about the hypotenuse. Section 2.2 asks you to try it and see what happens.

1.2 Calibration

Every question below should be answered for **two calibrations**, which differ only in σ_B :

Parameter	Symbol	Complements	Substitutes
Weight on food	α	0.60	0.60
Weight on the bus	β	0.50	0.50
Substitution, food vs. travel	σ_A	0.80	0.80
Substitution, bus vs. train	σ_B	0.40	3.00
Price of food	p_1	1.0	1.0
Price of a bus trip	p_2	1.0	1.0
Price of a train trip	p_3	1.5	1.5
Income	I	10.0	10.0

Hint: Write your code so that running everything again for the second calibration costs you one extra line, not a second copy of the code. In `Consumer.py` a consumer is created with `model = ConsumerClass(par={'sigma_B':3.0})`.

1.3 How do you know your answer is right?

There is **no formula** for the solution of this model, so you cannot look the answer up. That is the normal situation in model analysis, and it means you have to build your own confidence in the code. Three cheap checks, which you should use throughout the project:

1. Is the answer **possible**? All three shares should be strictly between 0 and 1 and sum to one, and all three quantities should be positive.
2. Do **two different methods** agree? In section 2 you solve the same problem twice, with two methods that have nothing in common.
3. Does the answer **stop moving** when you make the method more accurate? A finer grid, a tighter tolerance, another starting point.

Hint: Write these as `assert` statements, as in the Solow lecture, and re-run them every time you change your code. A broken test points straight at the bug – a wrong figure does not.

2 Solving the model numerically

We solve eq. (4) twice, in two very different ways.

2.1 A two-dimensional grid search

The idea is the one from the optimization lecture: lay a grid over everything the consumer can afford, compute utility in every point, and keep the best one. Nothing clever happens – and that is the point.

1. Make a grid of N values of s_1 and N values of w , both from 0 to 1, compute utility in every combination, and keep the best point. Implement it as `solve_grid()`. Report both (s_1, w) and the three budget shares it implies.

Hint: `np.meshgrid(s1_vec, w_vec, indexing='ij')` builds the grid, and `np.argmax` together with `np.unravel_index` finds the best point. Note that there is nothing to mask out and no `np.nan` anywhere: every point of the square is a possible choice.

2. Plot utility over the nested shares **in 3D** (`ax = fig.add_subplot(1,2,1,projection='3d')` and `ax.plot_surface()`) and as a **contour plot** (`ax.contourf()`), with the solution marked.
3. Try $N = 50, 100, 500$ and 1000 . How much does the answer move when you add points, and how many evaluations of u does it cost? Does it improve smoothly?
Hint: A grid can be lucky, see the optimization lecture. Judge from several values of N , never from one.

4. Compare the two calibrations. Where is the surface steep, and where is it flat? Which of the two would you expect to be harder for an optimizer, and why?

Note: Unlike in the lecture, there is no infeasible region to throw away here, and every point on your grid spends exactly the whole income. The nested budget shares bought us that for free.

2.2 L-BFGS-B

L-BFGS-B is the workhorse for smooth problems with several variables and simple bounds. The **B** stands for bounds: it takes a box around each variable, but it does

not take general constraints. Because we wrote the problem in nested budget shares, a box is all we need – the whole of `solve()` is

```
minimize(self.objective,s0,method='L-BFGS-B',bounds=((0,1),(0,1)))
```

and `objective()` is just $-u$. There is nothing else to arrange.

1. Solve the problem this way and implement it as `solve()`. Compare the solution, the number of function evaluations and the running time with the grid search. Do the two methods agree, and which of them do you trust more?
2. Record the **convergence path** with the callback argument of `optimize.minimize`, and draw it on top of the contour plot from section 2.1. Then plot how far each iteration is from the point the solver ends at, against the iteration number, on a **log scale**. Where does the method make fast progress, and where is it slow?

Hint: `callback=lambda sk: path.append(sk.copy())`. The `.copy()` is not optional – without it you get the same point repeated.

3. Solve from at least six different starting points, including the middle $(\frac{1}{2}, \frac{1}{2})$ and all four **corners** of the square. Do you always get the same answer? Does the number of iterations change?

One corner fails in one of the two calibrations, and reports `success = True` while doing so. Find it, and explain in economic terms what is special about that point and why the same start is fine in the other calibration. What is the practical rule to take away about where to start an optimizer?

4. **Why the parametrization matters.** `solve_flat_shares()` solves the same model the obvious way instead: it chooses the shares s_1 and s_2 directly, lets $s_3 = 1 - s_1 - s_2$ be the residual, and puts the same bounds $[0, 1]$ on both. Nothing is wrong with the code. Run it from $(\frac{1}{3}, \frac{1}{3})$, $(\frac{1}{2}, \frac{1}{2})$ and $(0.8, 0.8)$ in **both** calibrations and compare with your answer from question 1.

- (a) Look at s_3 in the answers. What has the solver done that the consumer cannot do?
- (b) What does `res.success` say in each case?
- (c) In one of the calibrations the solver reports a **higher** utility than the correct answer from question 1. How is that possible, and why is it in fact a useful warning sign?

- (d) The two calibrations go wrong in two different ways. Explain both, and relate them to whether bus and train are complements or substitutes.

Hint: The bounds $[0, 1]^2$ are no longer the set of possible choices. Which points does the box contain that the consumer cannot afford, and what happens to utility out there when the missing good is a complement, and when it is a substitute? This is the same lesson as converged = True in section 5.4 of the optimization lecture.

5. Vary the settings below – they all go in options={...} – and report the two shares, the utility reached and the number of function evaluations for each of them in one table:

(a) ftol (stop when the objective stops changing): $10^{-4}, 10^{-8}, 10^{-12}, 10^{-16}$

(b) gtol (stop when the gradient is small): $10^{-2}, 10^{-5}, 10^{-8}, 10^{-12}$

(c) eps (the step used to compute the gradient): $10^{-2}, 10^{-4}, 10^{-8}, 10^{-10}, 10^{-12}$

6. Which settings would you use? Note that you can rank them without knowing the true answer: we are maximizing, so a **higher utility is a better answer**. Why is a tighter tolerance not always a better answer, and why does eps have a best value rather than a smallest one?

Hint on eps: it is the size of the nudge in $u'(x) \approx \frac{u(x+\epsilon) - u(x)}{\epsilon}$. Both a large and a small value hurt, for two different reasons.

3 Relative prices and demand

We now use the model. Train tickets get more expensive, and we ask what the consumer does.

1. Solve the model for p_3 from 0.5 to 3.0, holding p_1 , p_2 and I fixed, and plot the three **budget shares** and the three **quantities**. Do it for both calibrations, ideally in the same figure.
2. Check the obvious first: does x_3 fall when p_3 rises, in both calibrations? It should.
3. In which calibration does a higher p_3 **raise** the budget share of the train, and in which does it **lower** it? Explain both cases.

Hint: Spending on trains is $p_3 x_3$. The price goes up and the quantity goes down, so which of the two wins? That is exactly what σ_B decides. Where is the borderline?

4. What happens to the number of **bus** trips when train tickets get more expensive? Explain why the two calibrations differ, and say which of them matches how you would react yourself.
5. How much does the price of a train ticket change the demand for **food**? Compare with the effect on bus trips, and relate the difference to the nest structure.
6. Repeat question 1 for the income I from 5 to 20 instead of p_3 . What happens to the three budget shares, and is that a reasonable description of how people behave?

4 Lump-sum taxes and product taxes

The government needs revenue, and has two kinds of instrument:

- a **lump-sum tax** T , which reduces income to $I - T$ and raises the revenue T
- **product taxes** $\tau_j \geq 0$, which raise the price of good j from p_j to $(1 + \tau_j)p_j$ and raise the revenue $\tau_j p_j x_j$

Total tax revenue is

$$R = T + \sum_{j=1}^3 \tau_j p_j x_j^*, \quad (5)$$

where x_j^* is what the consumer buys **given the taxes**. The consumer gets nothing back: the revenue leaves the model.

We compare the instruments in the simplest possible way. Two taxes that raise **the same revenue** are compared by the **utility** the consumer is left with. Higher is better.

Use `GovernmentClass` in `Government.py`. Its `set_taxes()` translates the taxes into the prices and the income the consumer faces, so everything you wrote in sections 1-3 keeps working unchanged.

1. Implement `tax_revenue()` (eq. (5)). Check it: a lump-sum tax T must raise revenue exactly T , and a zero tax must raise zero.
2. Consider five product taxes: on food only, on the bus only, on the train only, the same rate on bus and train, and the same rate on all three goods. For each of them, plot the revenue as a function of the tax rate for rates from 0 to 3, one panel per calibration.
3. Which of the curves have a **top**, and which just keep rising? Find the revenue-maximizing rate and the largest possible revenue where there is a top. In particular: what is the most that a tax on train tickets alone can ever raise in the substitutes calibration?

Hint: Use `optimize.minimize_scalar(...,method='bounded')` on minus the revenue – but not on the whole interval from 0 to 1000. At very high tax rates the consumer has given the good up completely, revenue is flat at almost nothing, and the search gets stuck out there and reports a maximum of zero. Read the top off your figure from question 2 first, and give the solver a bracket around it. This is the same point as the bracket in question 8 below.

4. Plot the consumer's utility **against the revenue raised**, for all five product taxes and for the lump-sum tax, one panel per calibration. Which instrument is the cheapest way to raise a given revenue, and does the ranking depend on the calibration?
5. Let the government need $R = 0.20$, i.e. 2 percent of income. Use a root-finder to find the tax rate that raises exactly that for each instrument, and report the rate and the resulting utility in one table per calibration. Rank the instruments, and comment in particular on the tax on train tickets.

Hint: `optimize.root_scalar(f, bracket=[...], method='brentq')` where f is the revenue minus the target.

6. Compare the **lump-sum tax** raising $R = 0.20$ with the uniform tax on **all three** goods raising $R = 0.20$. Is the consumer equally well off? Explain why, using the fact that a uniform tax leaves all the relative prices unchanged.
7. Now suppose the government needs $R = 0.50$ instead. Which instruments can deliver it, in each calibration? What does your root-finder do when the target cannot be reached, and how should your code handle that?
8. In the substitutes calibration there are **two** tax rates on bus trips that both raise $R = 0.20$. Find both. Which one should the government choose, and what is the difference between the two in terms of what the consumer does?

Hint: Two roots means two different brackets, and the bracket is part of the answer – read them off your figure from question 2. And **always check that a root really is a root**: `assert np.isclose(R, R_target)`. A root-finder returns a number whether or not it found anything, and here the revenue it is given is itself the output of a numerical solve, so it is only as accurate as that solve. One line of checking is cheap.

Finally, sum up: **what makes a product tax a cheap or an expensive way to raise revenue?** Your answer should say something about how large the taxed good is in the budget, and about how easily the consumer can avoid the tax by buying something else – and it should explain why taxing bus and train at the same rate is so much cheaper than taxing only one of them.

5 Extension

Extend the model in **one** direction of your own choice, solve it, and explain what new insight it gives. For example, you could:

1. Add a fourth good, either in the travel nest or in a nest of its own.
2. Require a minimum amount of food before anything else, so that utility depends on $x_1 - \underline{x}_1$. What does that do to the results in sections 3 and 4?
3. Let the consumer also choose how much to **work**, so income is earned rather than given, and redo the comparison in section 4.
4. Let there be **many consumers** with different incomes, and ask who ends up paying for a tax on travel.
5. Let the government **spend** the revenue on something the consumer values, and look for the combination (τ_1, τ_2, τ_3) that makes the consumer as well off as possible while still raising a given revenue. Which solver do you need now, and why is it no longer L-BFGS-B?
6. Replace the product tax with a tax of a fixed amount per unit bought, or with one that only applies above a certain quantity. The last one puts a kink in the problem – which methods survive it?

Note: This is intended as an open question, and we encourage you to be creative.